

Relatório de Segurança dos Contratos Inteligentes: Protocolo ARCA

Data: 20 de Abril de 2026

Autor: Manus AI

Projeto: Protocolo ARCA — Web3 MVP

Este documento apresenta uma análise detalhada das práticas de segurança implementadas nos contratos inteligentes que compõem o Protocolo ARCA (ARCAToken , ARCABadge , ARCASTaking e ARCAGovernance). A análise foca nas vulnerabilidades comuns em Solidity e como a arquitetura do protocolo as mitiga.

1. Visão Geral da Arquitetura de Segurança

O Protocolo ARCA foi desenvolvido utilizando a versão 0.8.24 da linguagem Solidity. Esta escolha é fundamental para a segurança base do sistema, pois a partir da versão 0.8.0 , o compilador Solidity inclui verificações nativas contra *overflow* e *underflow* aritméticos. Isso elimina a necessidade de bibliotecas externas como a SafeMath , reduzindo a complexidade do código e o custo de *gas*, mantendo a integridade das operações matemáticas em todo o protocolo.

Além disso, o protocolo faz uso extensivo da biblioteca de contratos inteligentes da OpenZeppelin, um padrão da indústria amplamente auditado e testado em produção. A herança de contratos como ERC20 , ERC721 , Ownable e ReentrancyGuard garante que as funcionalidades principais sigam as melhores práticas de segurança estabelecidas pela comunidade Ethereum.

2. Análise por Contrato

2.1. ARCAToken (ERC-20)

O contrato ARCAToken atua como o token de utilidade e governança do ecossistema.

Práticas de Segurança Implementadas:

- Padrão Consolidado:** Herda diretamente do contrato ERC20 da OpenZeppelin, garantindo que funções críticas como transfer , approve e transferFrom estejam imunes a vulnerabilidades conhecidas em implementações customizadas.
- Controle de Emissão (Minting):** A função mint é protegida pelo modificador onlyOwner , garantindo que apenas o administrador do protocolo (ou um contrato de

governança futuro) possa criar novos tokens. Isso previne a inflação descontrolada do token por atores mal-intencionados.

- **Fornecimento Inicial Fixo:** O construtor define um fornecimento inicial (mintado para o *deployer*), estabelecendo uma base econômica clara antes de qualquer emissão adicional.

2.2. ARCABadge (ERC-721)

O contrato `ARCABadge` gerencia a reputação das Organizações da Sociedade Civil (OSCs) através de NFTs não transferíveis (Soulbound Tokens).

Práticas de Segurança Implementadas:

- **Controle de Acesso Estrito:** A função `mintBadge` é restrita pelo modificador `onlyOwner`. Apenas a entidade governante pode atestar a reputação de uma OSC e emitir o badge correspondente, prevenindo a falsificação de credenciais.
- **Prevenção de Duplicidade:** A função `mintBadge` verifica ativamente se o endereço da OSC já possui um badge (`require(!possuiBadge[osc], "OSC ja possui um badge")`). Isso garante a unicidade da identidade on-chain de cada organização.
- **Soulbound Tokens (SBT):** O contrato sobrescreve as funções de transferência padrão do ERC-721 (`transferFrom` e `safeTransferFrom`), revertendo qualquer tentativa de transferência com a mensagem "Badges são intransferíveis (Soulbound)". Isso é crucial para o modelo de reputação, pois impede que uma OSC venda ou transfira seu histórico de confiabilidade para terceiros.

2.3. ARCAStaking

O contrato `ARCAStaking` permite que os usuários bloqueiem seus tokens ARC em troca de recompensas dinâmicas.

Práticas de Segurança Implementadas:

- **Proteção contra Reentrância:** As funções `stake` e `unstake` utilizam o modificador `nonReentrant` do `ReentrancyGuard` da OpenZeppelin. Isso previne ataques onde um contrato malicioso poderia chamar repetidamente a função de saque antes que o saldo interno fosse atualizado, drenando os fundos do contrato.
- **Padrão Checks-Effects-Interactions:** O contrato segue rigorosamente este padrão. Na função `unstake`, por exemplo, o saldo do usuário é zerado (Effect) *antes* da transferência do token ser realizada (Interaction). Isso adiciona uma camada extra de segurança contra reentrância.
- **Oráculo Descentralizado:** A taxa de recompensa é calculada com base no preço do ETH fornecido por um oráculo da Chainlink (`AggregatorV3Interface`). O uso de um oráculo

descentralizado mitiga o risco de manipulação de preços que ocorre frequentemente quando se utilizam *Automated Market Makers* (AMMs) locais como única fonte de dados.

- **Validação de Dados do Oráculo:** A função `getPrecoETH` verifica se o preço retornado pelo oráculo é maior que zero (`require(preco > 0, "Preco invalido do oraculo")`), protegendo o contrato contra falhas temporárias ou dados corrompidos do feed da Chainlink.

2.4. ARCAGovernance

O contrato `ARCAGovernance` implementa a lógica de Organização Autônoma Descentralizada (DAO) para o protocolo.

Práticas de Segurança Implementadas:

- **Prevenção de Spam:** A função `criarProposta` exige que o proponente possua um saldo mínimo de tokens ARC (`SALDO_MINIMO_PROPOSTA`). Isso atua como uma barreira econômica contra ataques de negação de serviço (DoS) que tentariam inundar a DAO com propostas irrelevantes.
- **Proteção contra Voto Duplo:** O contrato utiliza um mapeamento aninhado (`mapping(uint256 => mapping(address => bool)) public votou`) para registrar quem já participou de uma votação. A função `votar` verifica este mapeamento (`require(!votou[propostalId][msg.sender], "Voto duplicado")`), garantindo que cada endereço vote apenas uma vez por proposta.
- **Janelas de Tempo Rígidas:** As propostas possuem um período de votação estrito (`PERIODO_VOTACAO`). A função `votar` só permite interações enquanto o período estiver ativo, e a função `finalizarProposta` só pode ser chamada após o encerramento do prazo, garantindo a integridade do processo democrático.
- **Execução Segura:** A função `executarProposta` é protegida pelo `nonReentrant` e verifica se a proposta foi efetivamente aprovada e se ainda não foi executada, prevenindo execuções múltiplas ou não autorizadas.

3. Conclusão

O Protocolo ARCA demonstra um forte compromisso com a segurança de contratos inteligentes. A combinação do compilador Solidity atualizado, a utilização de bibliotecas padrão da indústria (OpenZeppelin), a implementação de proteções contra reentrância, o uso de oráculos descentralizados (Chainlink) e a aplicação de padrões de design seguros (Checks-Effects-Interactions) criam uma base robusta e resiliente contra os vetores de ataque mais comuns no ecossistema Web3.

Embora nenhuma auditoria possa garantir 100% de segurança, a arquitetura atual do Protocolo ARCA segue as melhores práticas recomendadas para o desenvolvimento de

aplicações descentralizadas (dApps).